

S-Kubed: Small Satellite Stationkeeping

Raygoza, Branden
011111841

Dr. Marco Maggia
Aerospace Engineering
May 18, 2026

1. How long did it take to complete this executive summary? 14 hours
2. Did you attend or watch the lesson provided by Mr. Paul Hottinger regarding Information Literacy (check the correct box)? ☐ YES ☒ NO

S-Kubed: Small Satellite Stationkeeping

Introduction

S-Kubed, or Small Satellite Stationkeeping, holds the primary goal of developing a fully functional test bed for small satellites, starting with CPP's Cerberus (a 3U project held by CUBEStep), and continuing with other companies' satellites as technologies and methodologies come into fruition. One of the required testing aspects is validating the in-orbit attitude determination and control subsystem (ADCS). But performing this test on the flight CubeSat risks damaging costly hardware. Because this test depends on rotational behavior of a rigid body, which is primarily governed by its moment of inertia (MOI) tensor, the testbed calls for a model that accurately recreates and represents these values to be tested before any irreplaceable equipment is verified.

The objective of this portion of the project was to design and build a dynamically representative physical model of the 3U CubeSat in the form of a solid, unchanging body whose inertia tensor can be configured to match a targeted value. To accomplish this, a software is also needed to determine how to realize this physical requirement. This model is used to reproduce the rotational dynamics of the spacecraft to be tested and does not contain functional avionic components.

Further beneficial characteristics of such a model include a reconfigurable mass distribution, which allows for this model to serve any target inertia tensor and remains useful as the flight design matures instead of being a single-use figure. It being 3-D printed from inexpensive materials further aids accessibility as specialized fabrication is not needed.

Problems in creating such a model include the ability to faithfully and accurately recreate a spacecraft's inertia tensor calling for the ability to have masses suspended in the center of the model. Additionally, the inertia tensor having six independent components and a change in a single one affecting all the others at once means that ideal physical placement for the target cannot be achieved by solving a single equation, requiring numerical optimization. This report describes the hardware and optimization software, presents validation results demonstrating targets are reached within 1% error, and identifies the principal limitations and recommended improvements.

Literature Review

An alternate perspective for a concept design was modeled by another teammate. Both are compared here as a trade study.

The initial concept used two printed structures with each containing a four-by-two arrangement of eight vertical rails. Mass blocks were threaded and affixed by screws, with a maximum of five blocks per rail and a total possible block count of 80. This was split into two because the available 3D printer is incapable of printing the full model height as a continuous part. The defining characteristic of this version is that each block's vertical coordinate is continuously adjustable.

The concept adopted by the team uses three printed structures, each containing five stacking frames configured as a five-by-five pocket grid. Mass blocks are printed with no clearance for a snug friction fit. It uses three units for uniformity and symmetry across the model. The project supervisor also stated that this improves manufacturability and accurately reflects standard production practice. The meaningful difference here is how the blocks are positioned.

The difference in block positioning is significant for the optimization process. Allowing for continuous vertical coordinate variability means that the software must determine the exact height of each block. Because a block's effect on the overall spin of the model grows with the mathematical square of its distance from the center, the relationship between the placement of the block and its effect is not linear, but quadratic. The software therefore cannot calculate it, but must search for it, and this may result in a solution that is accurate but is not optimal. Fixing every block into a known pocket removes this difficulty, as the effect of each pocket is fixed and known in advance. This means the software would not have to calculate every single possible solution in this continuous domain, but instead it must choose which pockets optimize the results with a fixed, discrete amount of possibilities. Furthermore, the fixed stacking frame version is more repeatable, as a screw-clamped block can slip or vary slightly in height, while a block seated in a pocket returns to the exact location every time, ensuring positive retention. This simpler calculation, improved repeatability, and easier manufacturing were the reasons for why the fixed-grid design was selected.

Another competing perspective regarded what the optimization should solve for. The initial model that was being developed by a different teammate used a separate code that optimized placements of masses for achieving a targeted center of mass (COM) instead of the inertia tensor. This reflects the impression that the COM is what defines a dynamically representative model. However, it is not adopted here because it does not govern rotational dynamics; two bodies can share identical COMs but rotate differently with the same torque if their inertia tensors are different. Therefore, the chosen model was instead built from scratch to match the six components of the inertia tensor, which is consistent with the faculty advisor's direction that moment of inertia accuracy is priority over center of mass and total mass.

Methodology

The methodology consists of a hardware architecture and an optimization program that use the same coordinate system, where X is the model's width, Y is the stacking axis, and Z is the model's depth, where inertia is expressed about the center of mass.

The hardware is built from three repeated 10 cm cubic units bonded with epoxy to form the 3U figure of the Cerberus spacecraft. Each unit consists of an open-topped box housing, a lid, five stacking frames, and a set of mass blocks. The housing walls are 6 mm thick to accommodate the heated thread inserts for securing the lid, which need material around the area to prevent it from shifting during installation. Each stacking frame is a five-by-five grid printed with 2mm wall thickness, using a triangular support geometry for rigidity. The mass blocks are open-topped boxes with walls that are thinner than the frame structure to allow for more mass to fit inside of it. The blocks are filled with lead pellets (chosen for its cost-efficiency in comparison to more costly alternatives like steel or tungsten cubes) to 14 grams and are held in place with one gram of epoxy resin. The inertia of the printed shell is derived from the SolidWorks mass-properties

report and is scaled by the ratio of the measured-printed to SolidWorks-reported mass. Because 3D printed models are not uniformly dense, this is a first order approximation.

The mass placement problem is solved in MATLAB. To solve for all six inertia tensor components at once, the software initializes a random block configuration and measures how far the current arrangement is from the target within its specified tolerance, and then steadily adjusts the blocks to minimize the score as much as possible, using the built-in MATLAB solver, `lsqnonlin` and its trust-region-reflective method. This means that the solver makes small careful changes, taking larger steps when they decrease error and smaller steps when they increase error. The effect of each block is calculated using the parallel-axis theorem, a standard physics formula, with a worked example present in Appendix B. To minimize the number of blocks, the software then runs a clean up process where it discourages the use of numerous tiny blocks and leans towards fewer heavier ones, a technique called reweighted L1 regularization. The next step repeatedly drops the lightest remaining blocks a few at a time and rechecks the configuration at each iteration to make sure the tolerance is satisfied. Finally, the software checks to see how the removal of each remaining block affects the result; if the block removal still allows for the accuracy target to be met, it will approve its removal, and if it does not, then it will replace it.

To avoid repetitive and unoptimized results if same starting locations were used for each search, the software instead runs the search 24 separate times from 24 random starting points and keeps the best result. Testing showed that increasing from 12 to 24 runs increased accuracy while anything beyond 24 did not. The solution with the fewest blocks is the one that is chosen because it simplifies physical assembly of the model. A safeguard that was implemented ensures that the requested target is physically possible before the software begins and stops with a clear message stating it is not possible, instead of searching endlessly for something that does not exist.

Results and Discussion

The software was validated using five test cases, ranging from a simple starting check to a full-sized realistic target. The complete set is listed in Table 1 of Appendix A. In the first test, the target was set equal to the model's own empty weight distribution; the software correctly added no blocks at all, which confirmed that it accurately understood the empty model was the starting point. The second test requested a symmetric target, the software produced a block arrangement in which the X and Z directions were nearly identical, as expected, which confirmed the parallel-axis theorem is applied consistently. The third test requested a lopsided target that resulted in blocks placed off-center in the expected direction to reflect the testing of non-zero diagonals, confirming that the software reacts correctly to uneven targets instead of repeating a single fixed pattern. The fourth test specified a target slightly different from the empty model and the software used only five blocks which confirmed that it only adds what is necessary. The fifth test was a full-sized realistic target that proved production standards could be satisfied, as all six MOI tensor components were within the specified accuracy. Samples of the resulting block arrangements are shown as figures in Appendix A.

Conclusion

This project produced a working, low-cost, adjustable system capable of recreating a CubeSat's weight distribution for pre-flight testing. The software finds block arrangements that match a requested moment of inertia target, and testing across five cases confirmed its ability to do so

accurately to within 1%. The physical model holds these lead-filled blocks in place using 3D-printed trays.

Testing also revealed limitations and improvements. On the physical side, another teammate's error in printing configuration resulted in the trays being printed too large, and, in order to conserve filament, he decided to shorten the top tray and its blocks to ensure proper fitting. An appropriate solution would be to print a calibration cube from the designated 3D-printer, calculate the error in printing along the vertical dimension, and use that percentage to scale down the stacking frames prior to printing for retained symmetry. Also, the blocks are currently held in place by friction fitting alone and could be further secured by positive features like corner pins or rails. Labels can also be added to ensure proper orientation and tray numbers can be molded into the parts to ensure proper reproducibility. Furthermore, the blocks only reached 15 grams each, which limits how the weight distribution can be adjusted. Switching to machined steel blocks or tungsten blocks can raise the mass size to about 28 grams and 70 grams respectively. The software already supports heavier blocks with only two setting changes. On the software side, the plastic shell's properties are an estimate from the computer model and could instead be measured directly with a pendulum test. Lastly, the center of mass could be added as a target in the future if required by changing another software setting, at the cost of moment of inertia accuracy.

Works Cited

- Dassault Systèmes, “Mass Properties and Section Properties,” SOLIDWORKS Web Help, 2025.
URL:
https://help.solidworks.com/2025/english/solidworks/sldworks/t_adding_mass_properties.htm [retrieved 18 May 2026].
- The MathWorks, Inc., “lsqnonlin — Solve Nonlinear Least-Squares Problems,” MATLAB Optimization Toolbox Documentation, R2026a. URL:
<https://www.mathworks.com/help/optim/ug/lsgnonlin.html> [retrieved 18 May 2026].
- Coleman, T. F., and Li, Y., “An Interior Trust Region Approach for Nonlinear Minimization Subject to Bounds,” SIAM Journal on Optimization, Vol. 6, No. 2, 1996, pp. 418–445.
- Candès, E. J., Wakin, M. B., and Boyd, S. P., “Enhancing Sparsity by Reweighted L1 Minimization,” Journal of Fourier Analysis and Applications, Vol. 14, No. 5, 2008, pp. 877–905.

Appendix A: Tables and Figures

Table 1. *Validation test matrix summarizing the five test cases used to verify the optimization code*

Test	Target Inertia Tensor ($\text{g}\cdot\text{cm}^2$)	Blocks	Accuracy Result	Status
1	Target set equal to scaled static hardware properties (baseline)	0	All six components within 0.03%	PASS
2	$I_{xx} = I_{zz} = 120,000$; $I_{yy} = 33,000$; off-diagonals = 0	41	I_{xx} and I_{zz} agree to within 0.04%	PASS
3	Same diagonals as Test 2; $I_{xy} = -8,000$	42	Correct negative sign on I_{xy} ; within tolerance	PASS
4	Targets slightly above the static baseline	5	All components within 1% tolerance	PASS
5	$I_{xx} = I_{zz} = 150,000$; $I_{yy} = 35,000$ (production scale)	50	All components within 1% tolerance	PASS

=====

3U CUBESAT INERTIA OPTIMIZATION REPORT

Axes: X = width, Y = stacking (long), Z = depth

(MOI-prioritized; COM is informational)

=====

Static Hardware Mass: 832 g

Active Tuning Blocks: 52

Metric	Target	Actual	Diff	Error (%)	Status

Mass (g)	INFO	1612.3800	-	-	N/A
COM X (cm)	0.0000	0.0480	4.8004e-02	-	INFO
COM Y (cm)	0.0000	0.3835	3.8347e-01	-	INFO
COM Z (cm)	0.0000	0.0268	2.6840e-02	-	INFO
I _{xx} (g*cm^2)	150000.0000	148804.6116	1.1954e+03	0.797%	PASS
I _{yy} (g*cm^2)	35000.0000	34923.3430	7.6657e+01	0.219%	PASS
I _{zz} (g*cm^2)	150000.0000	148578.6527	1.4213e+03	0.948%	PASS
I _{xy} (g*cm^2)	-3500.0000	-3377.4672	1.2253e+02	0.082%	PASS
I _{xz} (g*cm^2)	4000.0000	4173.4214	1.7342e+02	0.116%	PASS
I _{yz} (g*cm^2)	5000.0000	5319.5315	3.1953e+02	0.213%	PASS

--- ASSEMBLED INERTIA TENSOR (g*cm^2) ---

X	Y	Z
X [148804.61	-3377.47	4173.42]
Y [-3377.47	34923.34	5319.53]
Z [4173.42	5319.53	148578.65]

--- BLOCK PLACEMENT (Active Slots Only) ---

Mass (g)	X (cm)	Y (cm)	Z (cm)

15	1.72	-13.42	-3.44
15	3.44	-13.42	-3.44
15	1.72	-11.66	-3.44
15	3.44	-11.66	-3.44

Figure 1. Sample of MATLAB code summary table output using realistic target diagonals with non-zero off-diagonals; List of block placement continues for all 52 blocks

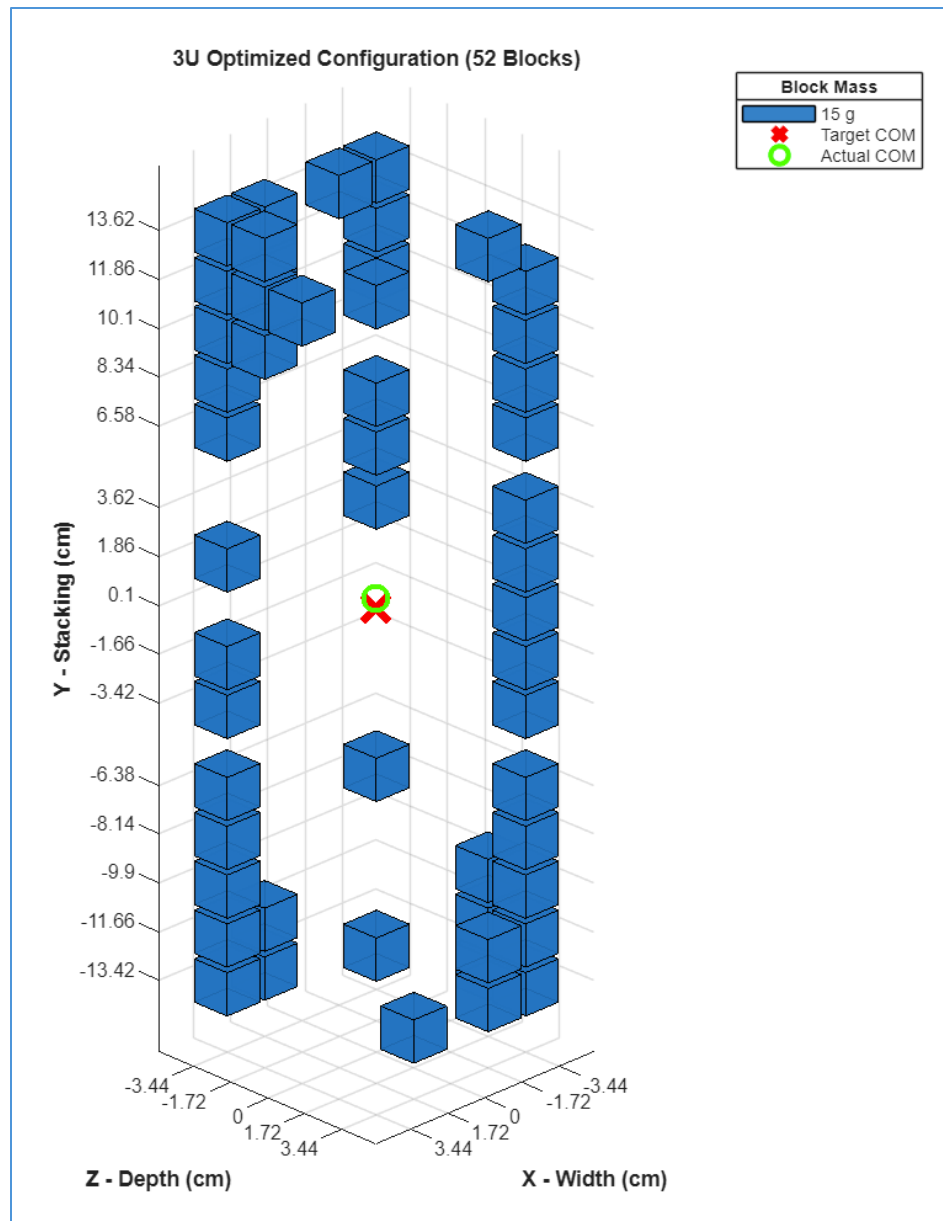


Figure 2. *MATLAB code block placement sample for configuration demonstrated in Figure 1 above*

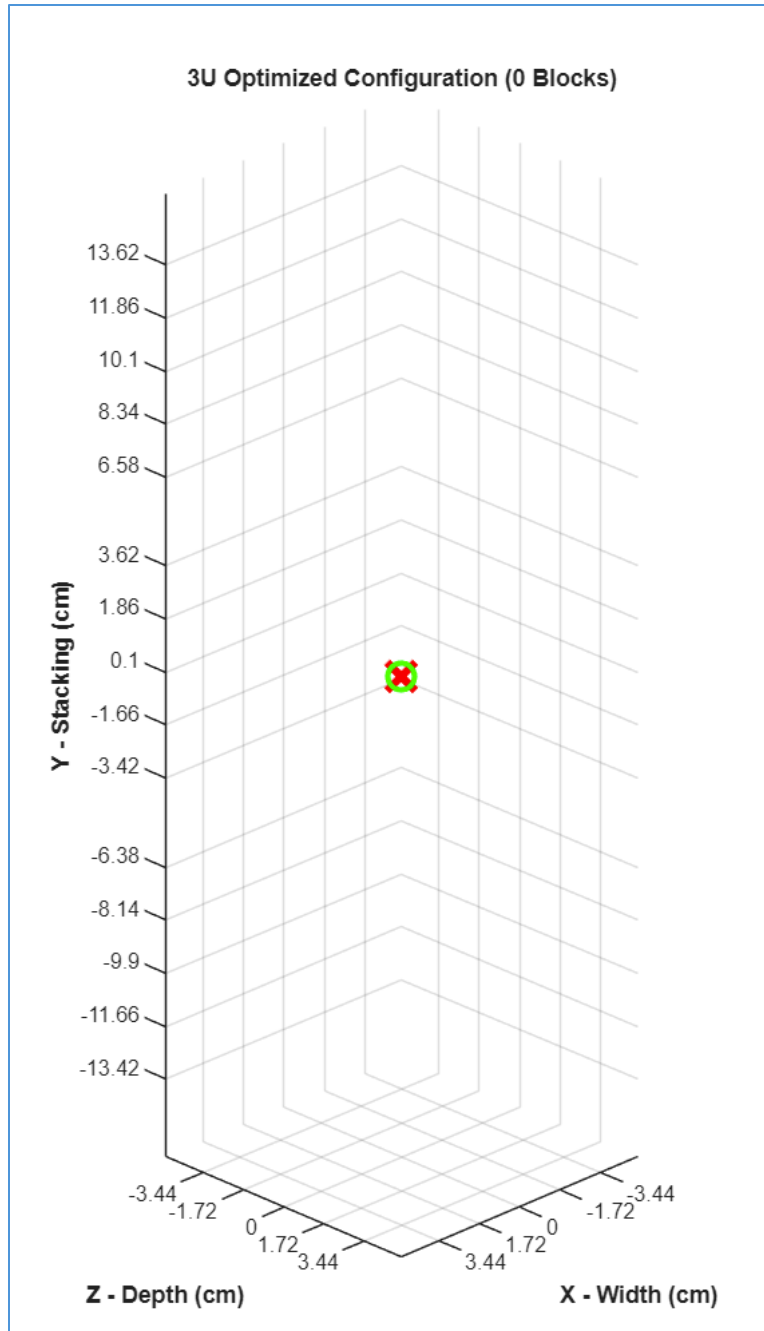


Figure 3. *First test output, demonstrating accurate SolidWorks mass-properties implementation*

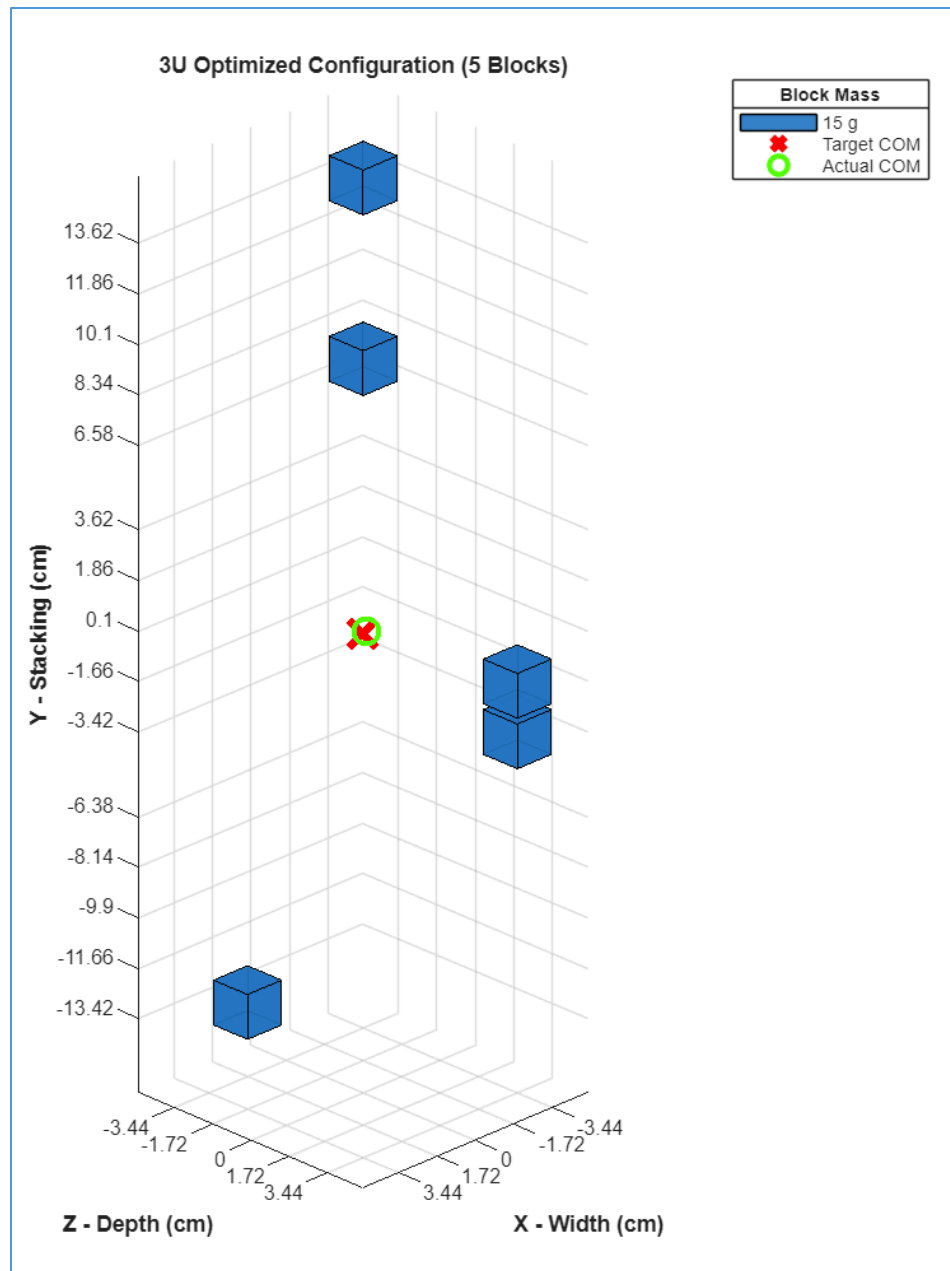


Figure 4. Test 4 results demonstrating minimal blocks required for achieving a moment of inertia that is slightly above the static line

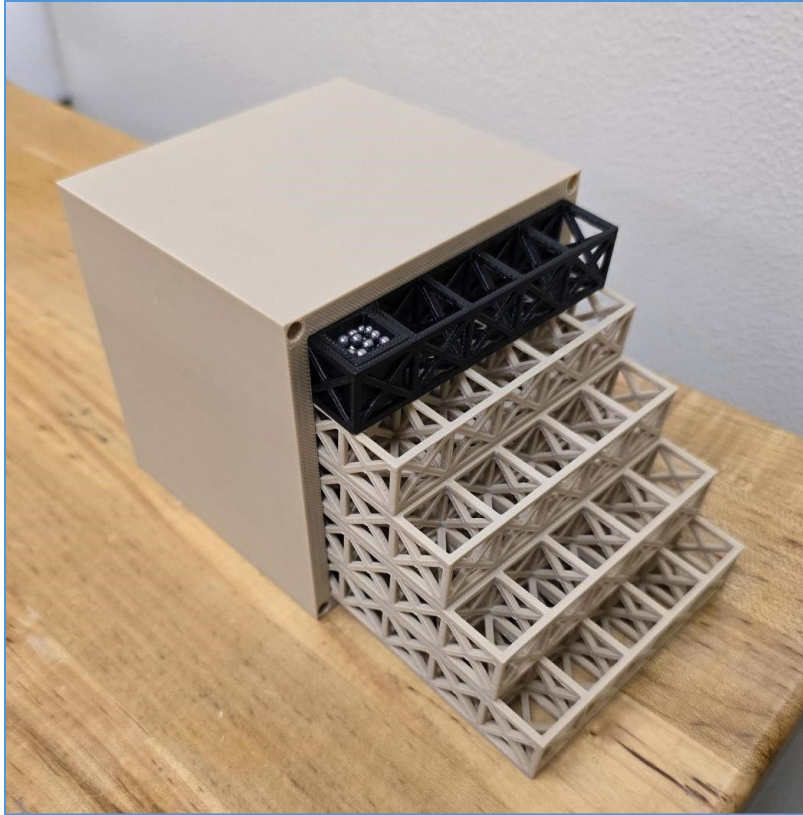


Figure 5. *First 1U CubeSat prototype with stacking frames and lead-filled blocks*

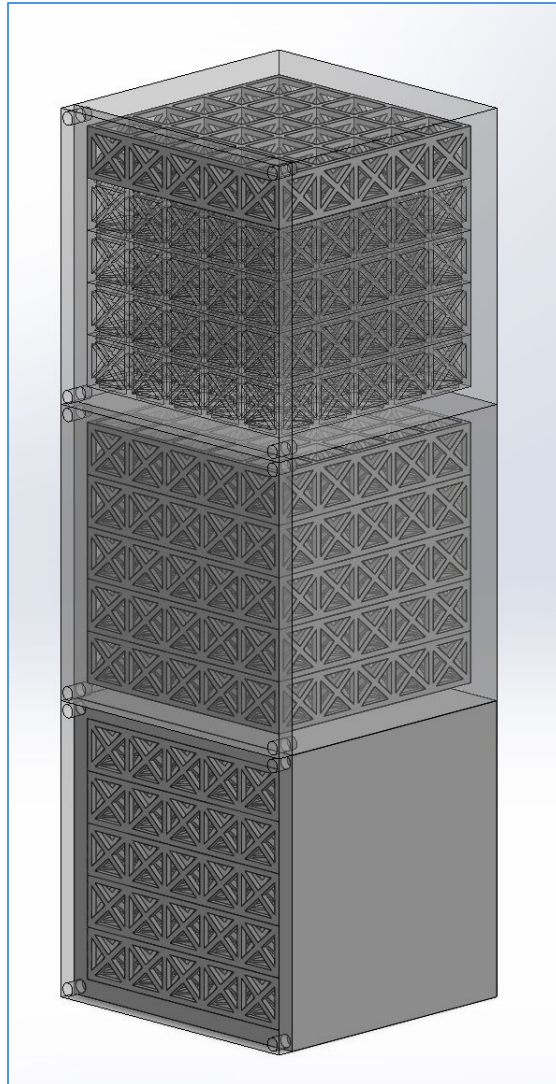


Figure 6. *3U SolidWorks configuration*

Appendix B: Sample Calculation

Parallel-Axis Inertia Contribution of a Single Mass Block

This calculation demonstrates how the optimization code computes the inertia contribution of one mass block placed at a grid location. Each block is treated as a point mass located at its centroid, and its contribution to the inertia tensor is found using the parallel-axis theorem.

For a point mass m located at position (x, y, z) measured from the model center of mass, the six tensor contributions are:

$$I_{xx} = m (y^2 + z^2)$$

$$I_{yy} = m (x^2 + z^2)$$

$$I_{zz} = m (x^2 + y^2)$$

$$I_{xy} = -m \cdot x \cdot y$$

$$I_{xz} = -m \cdot x \cdot z$$

$$I_{yz} = -m \cdot y \cdot z$$

The values below are taken from the first active block listed in the optimization report: a 15 g block located at $(x, y, z) = (1.72, -13.42, -3.44)$ cm, measured from the model center of mass. Given: $m = 15$ g ; $(x, y, z) = (1.72, -13.42, -3.44)$ cm

$$I_{xx} = m (y^2 + z^2) = 15 [(-13.42)^2 + (-3.44)^2] = 15 (180.0964 + 11.8336) = 2878.95 \text{ g}\cdot\text{cm}^2$$

$$I_{yy} = m (x^2 + z^2) = 15 [(1.72)^2 + (-3.44)^2] = 15 (2.9584 + 11.8336) = 221.88 \text{ g}\cdot\text{cm}^2$$

$$I_{zz} = m (x^2 + y^2) = 15 [(1.72)^2 + (-13.42)^2] = 15 (2.9584 + 180.0964) = 2745.82 \text{ g}\cdot\text{cm}^2$$

$$I_{xy} = -m \cdot x \cdot y = -15 (1.72)(-13.42) = +346.24 \text{ g}\cdot\text{cm}^2$$

$$I_{xz} = -m \cdot x \cdot z = -15 (1.72)(-3.44) = +88.75 \text{ g}\cdot\text{cm}^2$$

$$I_{yz} = -m \cdot y \cdot z = -15 (-13.42)(-3.44) = -692.47 \text{ g}\cdot\text{cm}^2$$

These six values are the contribution of this single block to the inertia tensor. The optimization software repeats this calculation for every active block - 52 blocks in the run shown in Appendix A - and sums the corresponding components across all of them. Adding the scaled static-shell inertia to these block sums produces the assembled inertia tensor reported by the program. For example, the assembled I_{xx} of 148804.61 $\text{g}\cdot\text{cm}^2$ is the shell's I_{xx} plus the combined I_{xx} contributions of all 52 blocks. The optimizer adjusts which slots are filled, and with how much mass, until this assembled tensor matches the target tensor within the specified tolerance.

The products of inertia use the convention $I_{xy} = -m \cdot x \cdot y$, and likewise for I_{xz} and I_{yz} . A single block may contribute either sign; the negative off-diagonal totals in the assembled tensor (for example, $I_{xy} = -3377.47 \text{ g}\cdot\text{cm}^2$) are the net result of summing positive and negative contributions across all blocks.